LDL08H06A

2008 年 8 月 6 日

第 1 章

論理譜

```
{ ================================================================== }
{   並び替えプロセッサ                                               }
{ ================================================================== }
logicname LDL08H06A

{ ================================================================== }
{   実効譜                                                           }
{ ================================================================== }
entity main
{ ------------------------------------------------------------------ }
{   入力                                                             }
{ ------------------------------------------------------------------ }
input  reset;
input  portin[8];

{ ------------------------------------------------------------------ }
{   双方向                                                           }
{ ------------------------------------------------------------------ }
inout  data[8];

{ ------------------------------------------------------------------ }
{   出力                                                             }
{ ------------------------------------------------------------------ }
output address[4];
output cs;
output re;
output we;
output portout[8];
output writedata[8];

{ ------------------------------------------------------------------ }
{   内部信号                                                         }
{ ------------------------------------------------------------------ }
bitn   datain[8];
bitn   dataout[8];
bitn   datacnt;
bitr   codepoint[8];
bitn   codec[5];
bitn   coded[8];
bitr   regA[8];
bitr   regB[8];
bitr   regX[8];
bitr   flgC;
bitr   flgZ;
bitr   regport[8];
bitn   codec1p;
bitn   codec2p;
bitn   codec3p;
```

```
bitn   codec4p;
bitn   codec5p;
bitn   codec6p;
bitn   codec7p;
bitn   codec8p;
bitn   codec9p;
bitn   codec10p;
bitn   codec12p;
bitn   codec13p;
bitn   codec15p;
bitn   codec23p;
bitn   codec25p;
bitn   codec26p;
bitn   codec27p;
bitn   codec28p;
bitn   codec29p;
bitn   codec30p;

{ ------------------------------------------------------------------ }
{   検査端子                                                         }
{ ------------------------------------------------------------------ }
output T0P[8]; T0P=codepoint;
output T1P[8]; T1P=regB;
output T2P[8]; T2P=regA;
output T3P[8]; T3P=regX;
output T4P;    T4P=flgZ;
output T5P;    T5P=flgC;

{ ------------------------------------------------------------------ }
{   出力代入                                                         }
{ ------------------------------------------------------------------ }
   address=regX.0:3;
   writedata=dataout;
   we=datacnt;
   re=codec2p|codec5p|codec27p|codec28p;
   cs=codec1p|codec2p|codec5p|codec23p|codec26p|codec27p|codec28p;
   portout=regport;

{ ------------------------------------------------------------------ }
{   双方向定義                                                       }
{ ------------------------------------------------------------------ }
   enable(data,datain,dataout,datacnt)

   datain.8=1;

{ ------------------------------------------------------------------ }
{   双方向出力                                                       }
{ ------------------------------------------------------------------ }
   switch(codec)
      case 1:  dataout=regA;
      case 23: dataout=coded;
      case 26: dataout=regB;
   endswitch

{ ------------------------------------------------------------------ }
{   双方向制御                                                       }
{ ------------------------------------------------------------------ }
   if (codec1p|codec23p|codec26p) datacnt=1; endif

{ ------------------------------------------------------------------ }
{   プログラムカウンタ                                               }
{ ------------------------------------------------------------------ }
   if (reset)
      codepoint=0;
   else
```

```
      if (codec6p&flgC)               { JPC b8 }
         codepoint=coded;
      else
         if (codec7p&!flgC)           { JPNC b8 }
            codepoint=coded;
         else
            if (codec8p&flgZ)         { JPZ b8 }
               codepoint=coded;
            else
               if (codec9p&!flgZ)     { JPNZ b8 }
                  codepoint=coded;
               else
                  if (codec10p)       { JP b8 }
                     codepoint=coded;
                  else
                     codepoint=codepoint+1;
                  endif
               endif
            endif
         endif
      endif
   endif

{ ------------------------------------------------------------------ }
{   プログラム                                    }
{ ------------------------------------------------------------------ }
   switch(codepoint)
      case 0:  codec=0;  coded=0;
      case 1:  codec=23; coded=17;  { LD (X),17 }
      case 2:  codec=4;  coded=1;   { LD X,1 }
      case 3:  codec=23; coded=16;  { LD (X),16 }
      case 4:  codec=4;  coded=2;   { LD X,2 }
      case 5:  codec=23; coded=15;  { LD (X),15 }
      case 6:  codec=4;  coded=3;   { LD X,3 }
      case 7:  codec=23; coded=14;  { LD (X),14 }
      case 8:  codec=4;  coded=4;   { LD X,4 }
      case 9:  codec=23; coded=13;  { LD (X),13 }
      case 10: codec=4;  coded=5;   { LD X,5 }
      case 11: codec=23; coded=12;  { LD (X),12 }
      case 12: codec=4;  coded=6;   { LD X,6 }
      case 13: codec=23; coded=11;  { LD (X),11 }
      case 14: codec=4;  coded=7;   { LD X,7 }
      case 15: codec=23; coded=10;  { LD (X),10 }
      case 16: codec=4;  coded=8;   { LD X,8 }
      case 17: codec=23; coded=9;   { LD (X),9  }
      case 18: codec=4;  coded=9;   { LD X,9 }
      case 19: codec=23; coded=8;   { LD (X),8  }
      case 20: codec=29;            { LD  A,PORT }
      case 21: codec=13; coded=0;   { CMP A,0 }
      case 22: codec=8;  coded=20;  { JPZ 20 }
      case 23: codec=4; coded=10;   { LD   X,10 }
      case 24: codec=28;            { LD   X,(X) }
      case 25: codec=2;             { LD   A,(X) }
      case 26: codec=15;            { INC  X }
      case 27: codec=5;             { CMP  A,(X) }
      case 28: codec=6; coded=49;   { JPC  49 }
      case 29: codec=27;            { LD   B,(X) }
      case 30: codec=1;             { LD   (X),A }
      case 31: codec=25;            { DEC  X }
      case 32: codec=26;            { LD   (X),B }
      case 33: codec=22;            { LD   A,X }
      case 34: codec=4; coded=11;   { LD   X,11 }
```

```
      case 35: codec=23; coded=255; { LD   (X),0FFh }
      case 36: codec=21;            { LD   X,A }
      case 37: codec=10; coded=49;  { JP   49 }
      case 49: codec=15;            { INC  X }
      case 50: codec=22;            { LD   A,X }
      case 51: codec=13; coded=9;   { CMP  A,9 }
      case 52: codec=6; coded=60;   { JPC  60   }
      case 53: codec=4; coded=11;   { LD   X,11 }
      case 54: codec=3; coded=0;    { LD   A,0 }
      case 55: codec=5;             { CMP  A,(X) }
      case 56: codec=8;  coded=70;  { JPZ  70 }
      case 57: codec=23; coded=0;   { LD   (X),0 }
      case 58: codec=3; coded=0;    { LD   A,0 }
      case 60: codec=4; coded=10;   { LD   X,10 }
      case 61: codec=1;             { LD   (X),A }
      case 62: codec=10; coded=23;  { JP   23 }
      case 70: codec=3;  coded=1;   { LD   A,1 }
      case 71: codec=30;            { LD   PORT,A }
      case 72: codec=10; coded=72;  { JP   72 }
   endswitch

{ ------------------------------------------------------------------ }
{   ポート                                                          }
{ ------------------------------------------------------------------ }
   if (reset)
      regport=0;
   else
      if (codec30p)
         regport=regA;
      else
         regport=regport;
      endif
   endif

{ ------------------------------------------------------------------ }
{   レジスタ A                                                      }
{ ------------------------------------------------------------------ }
   if (reset)
      regA=0;
   else
      switch(codec)
         case 2:  regA=datain;
         case 3:  regA=coded;
         case 14: regA=regA+1;
         case 20: regA=regB;
         case 22: regA=regX;
         case 24: regA=regA-1;
         case 29: regA=portin;
         default: regA=regA;
      endswitch
   endif

{ ------------------------------------------------------------------ }
{   レジスタ B                                                      }
{ ------------------------------------------------------------------ }
   if (reset)
      regB=0;
   else
      switch(codec)
         case 11: regB=regA;
         case 27: regB=datain;
         default: regB=regB;
      endswitch
   endif
```

```
{ ------------------------------------------------------------------ }
{   レジスタ X                                          }
{ ------------------------------------------------------------------ }
   if (reset)
      regX=0;
   else
      switch(codec)
         case 4:  regX=coded;
         case 15: regX=regX+1;
         case 21: regX=regA;
         case 25: regX=regX-1;
         case 28: regX=datain;
         default: regX=regX;
      endswitch
   endif

{ ------------------------------------------------------------------ }
{   フラグ C                                            }
{ ------------------------------------------------------------------ }
   if (reset)
      flgC=0;
   else
      if (codec5p)
         if (regA<datain)
            flgC=1;
         else
            flgC=0;
         endif
      else
         if (codec12p)
            if (regA<regB)
               flgC=1;
            else
               flgC=0;
            endif
         else
            if (codec13p)
               if (regA<coded)
                  flgC=1;
               else
                  flgC=0;
               endif
            else
               flgC=flgC;
            endif
         endif
      endif
   endif

{ ------------------------------------------------------------------ }
{   フラグ Z                                            }
{ ------------------------------------------------------------------ }
   if (reset)
      flgZ=0;
   else
      if (codec5p)
         if (regA==datain)
            flgZ=1;
         else
            flgZ=0;
         endif
      else
         if (codec12p)
            if (regA==regB)
```

```
                flgZ=1;
             else
                flgZ=0;
             endif
          else
             if (codec13p)
                if (regA==coded)
                   flgZ=1;
                else
                   flgZ=0;
                endif
             else
                flgZ=flgZ;
             endif
          endif
       endif
    endif

{ ------------------------------------------------------------------ }
{   命令指標                                                        }
{ ------------------------------------------------------------------ }
    switch(codec)
       case 1:  codec1p=1;   { LD (X),A }
       case 2:  codec2p=1;   { LD A,(X) }
       case 3:  codec3p=1;   { LD A,d8 }
       case 4:  codec4p=1;   { LD X,d8 }
       case 5:  codec5p=1;   { CMP A,(X) }
       case 6:  codec6p=1;   { JPC b8 }
       case 7:  codec7p=1;   { JPNC b8 }
       case 8:  codec8p=1;   { JPZ b8 }
       case 9:  codec9p=1;   { JPNZ b8 }
       case 10: codec10p=1;  { JP b8 }
       case 12: codec12p=1;  { CMP A,B }
       case 13: codec13p=1;  { CMP A,d8 }
       case 15: codec15p=1;  { INC X }
       case 23: codec23p=1;  { LD (X),d8 }
       case 25: codec25p=1;  { DEC X }
       case 26: codec26p=1;  { LD (X),B }
       case 27: codec27p=1;  { LD B,(X) }
       case 28: codec28p=1;  { LD (X),X }
       case 29: codec29p=1;  { LD A,PORT }
       case 30: codec30p=1;  { LD PORT,A }
    endswitch

ende
```

```
{ ================================================================ }
{   機能実行譜                                                       }
{ ================================================================ }
entity sim
{ ---------------------------------------------------------------- }
{   検査端子                                                         }
{ ---------------------------------------------------------------- }
output reset;
output portin[8];
output data[8];
output address[4];
output cs;
output re;
output we;
output portout[8];
output writedata[8];

{ ---------------------------------------------------------------- }
{   内部検査端子                                                     }
{ ---------------------------------------------------------------- }
output TB0P[8];
output TB1P[8];
output TB2P[8];
output TB3P[8];
output TB4P[8];
output TB5P[8];
output TB6P[8];
output TB7P[8];
output TB8P[8];
output TB9P[8];
output TB10P[8];
output TB11P[8];

{ ---------------------------------------------------------------- }
{   内部信号                                                         }
{ ---------------------------------------------------------------- }
bitr   tc[11];
bitr   memory0d[8];
bitr   memory1d[8];
bitr   memory2d[8];
bitr   memory3d[8];
bitr   memory4d[8];
bitr   memory5d[8];
bitr   memory6d[8];
bitr   memory7d[8];
bitr   memory8d[8];
bitr   memory9d[8];
bitr   memory10d[8];
bitr   memory11d[8];
bitn   node_cs;
bitn   node_re;
bitn   node_we;
bitn   node_address[4];
bitn   node_writedata[8];
bitn   node_reset;

{ ---------------------------------------------------------------- }
{   検査端子代入                                                     }
{ ---------------------------------------------------------------- }
   TB0P=memory0d;
   TB1P=memory1d;
   TB2P=memory2d;
   TB3P=memory3d;
```

```
   TB4P=memory4d;
   TB5P=memory5d;
   TB6P=memory6d;
   TB7P=memory7d;
   TB8P=memory8d;
   TB9P=memory9d;
   TB10P=memory10d;
   TB11P=memory11d;

{ ------------------------------------------------------------- }
{   メモリ選択                                                  }
{ ------------------------------------------------------------- }
   cs=node_cs;

{ ------------------------------------------------------------- }
{   メモリ読み出し                                              }
{ ------------------------------------------------------------- }
   re=node_re;

{ ------------------------------------------------------------- }
{   メモリ書き込み                                              }
{ ------------------------------------------------------------- }
   we=node_we;

{ ------------------------------------------------------------- }
{   メモリ番地                                                  }
{ ------------------------------------------------------------- }
   address=node_address;

{ ------------------------------------------------------------- }
{   メモリ書き込みデータ                                        }
{ ------------------------------------------------------------- }
   writedata=node_writedata;

{ ------------------------------------------------------------- }
{   実効譜導入                                                  }
{ ------------------------------------------------------------- }
   part main(reset,portin,data,node_address,node_cs,node_re
            ,node_we,portout,node_writedata)

{ ------------------------------------------------------------- }
{   検査位置                                                    }
{ ------------------------------------------------------------- }
   tc=tc+1;

{ ------------------------------------------------------------- }
{   初期化                                                      }
{ ------------------------------------------------------------- }
   if (tc<5) node_reset=1; endif

   reset=node_reset;

{ ------------------------------------------------------------- }
{   入力ポート                                                  }
{ ------------------------------------------------------------- }
   if (tc>35) portin=1; endif

{ ------------------------------------------------------------- }
{   メモリ 0                                                    }
{ ------------------------------------------------------------- }
   if (node_reset)
      memory0d=0;
   else
      if (node_cs)
```

```
         if (node_we)
            if (node_address==0)
               memory0d=node_writedata;
            else
               memory0d=memory0d;
            endif
         else
            memory0d=memory0d;
         endif
      else
         memory0d=memory0d;
      endif
   endif

{ ------------------------------------------------------------------ }
{   メモリ 1                                          }
{ ------------------------------------------------------------------ }
   if (node_reset)
      memory1d=0;
   else
      if (node_cs)
         if (node_we)
            if (node_address==1)
               memory1d=node_writedata;
            else
               memory1d=memory1d;
            endif
         else
            memory1d=memory1d;
         endif
      else
         memory1d=memory1d;
      endif
   endif

{ ------------------------------------------------------------------ }
{   メモリ 2                                          }
{ ------------------------------------------------------------------ }
   if (node_reset)
      memory2d=0;
   else
      if (node_cs)
         if (node_we)
            if (node_address==2)
               memory2d=node_writedata;
            else
               memory2d=memory2d;
            endif
         else
            memory2d=memory2d;
         endif
      else
         memory2d=memory2d;
      endif
   endif

{ ------------------------------------------------------------------ }
{   メモリ 3                                          }
{ ------------------------------------------------------------------ }
   if (node_reset)
      memory3d=0;
   else
      if (node_cs)
         if (node_we)
            if (node_address==3)
```

```
                memory3d=node_writedata;
             else
                memory3d=memory3d;
             endif
          else
             memory3d=memory3d;
          endif
       else
          memory3d=memory3d;
       endif
    endif

{ ------------------------------------------------------------------ }
{   メモリ 4                                          }
{ ------------------------------------------------------------------ }
    if (node_reset)
       memory4d=0;
    else
       if (node_cs)
          if (node_we)
             if (node_address==4)
                memory4d=node_writedata;
             else
                memory4d=memory4d;
             endif
          else
             memory4d=memory4d;
          endif
       else
          memory4d=memory4d;
       endif
    endif

{ ------------------------------------------------------------------ }
{   メモリ 5                                          }
{ ------------------------------------------------------------------ }
    if (node_reset)
       memory5d=0;
    else
       if (node_cs)
          if (node_we)
             if (node_address==5)
                memory5d=node_writedata;
             else
                memory5d=memory5d;
             endif
          else
             memory5d=memory5d;
          endif
       else
          memory5d=memory5d;
       endif
    endif

{ ------------------------------------------------------------------ }
{   メモリ 6                                          }
{ ------------------------------------------------------------------ }
    if (node_reset)
       memory6d=0;
    else
       if (node_cs)
          if (node_we)
             if (node_address==6)
                memory6d=node_writedata;
             else
```

```
            memory6d=memory6d;
         endif
      else
         memory6d=memory6d;
      endif
   else
      memory6d=memory6d;
   endif
endif

{ ------------------------------------------------------------------ }
{   メモリ 7                                          }
{ ------------------------------------------------------------------ }
if (node_reset)
   memory7d=0;
else
   if (node_cs)
      if (node_we)
         if (node_address==7)
            memory7d=node_writedata;
         else
            memory7d=memory7d;
         endif
      else
         memory7d=memory7d;
      endif
   else
      memory7d=memory7d;
   endif
endif

{ ------------------------------------------------------------------ }
{   メモリ 8                                          }
{ ------------------------------------------------------------------ }
if (node_reset)
   memory8d=0;
else
   if (node_cs)
      if (node_we)
         if (node_address==8)
            memory8d=node_writedata;
         else
            memory8d=memory8d;
         endif
      else
         memory8d=memory8d;
      endif
   else
      memory8d=memory8d;
   endif
endif

{ ------------------------------------------------------------------ }
{   メモリ 9                                          }
{ ------------------------------------------------------------------ }
if (node_reset)
   memory9d=0;
else
   if (node_cs)
      if (node_we)
         if (node_address==9)
            memory9d=node_writedata;
         else
            memory9d=memory9d;
         endif
```

```
         else
            memory9d=memory9d;
         endif
      else
         memory9d=memory9d;
      endif
   endif

{ ------------------------------------------------------------------ }
{   メモリ 10                                          }
{ ------------------------------------------------------------------ }
   if (node_reset)
      memory10d=0;
   else
      if (node_cs)
         if (node_we)
            if (node_address==10)
               memory10d=node_writedata;
            else
               memory10d=memory10d;
            endif
         else
            memory10d=memory10d;
         endif
      else
         memory10d=memory10d;
      endif
   endif

{ ------------------------------------------------------------------ }
{   メモリ 11                                          }
{ ------------------------------------------------------------------ }
   if (node_reset)
      memory11d=0;
   else
      if (node_cs)
         if (node_we)
            if (node_address==11)
               memory11d=node_writedata;
            else
               memory11d=memory11d;
            endif
         else
            memory11d=memory11d;
         endif
      else
         memory11d=memory11d;
      endif
   endif

{ ------------------------------------------------------------------ }
{   データバス入力                                     }
{ ------------------------------------------------------------------ }
   if (node_cs&node_re)
      switch(node_address)
         case 0:  data=memory0d;
         case 1:  data=memory1d;
         case 2:  data=memory2d;
         case 3:  data=memory3d;
         case 4:  data=memory4d;
         case 5:  data=memory5d;
         case 6:  data=memory6d;
         case 7:  data=memory7d;
         case 8:  data=memory8d;
         case 9:  data=memory9d;
```

```
         case 10: data=memory10d;
         case 11: data=memory11d;
      endswitch
   endif

ende

endlogic
```